

FEDERATED LEARNING PRIVACY AND SECURITY ENHANCEMENT

Oscar Chen
 Department of Computer Science
 Golisano College of Computing and Information Sciences
 Rochester Institute of Technology
 Rochester, NY 14586
 oyc2709@rit.edu

Abstract—Federated Learning (FL) has emerged as a promising alternative to centralized machine learning by enabling collaborative model training without direct data sharing. With recent advances in FL on feature analysis, aggregation algorithms, privacy, and security, FL improves data locality and lowers raw data exposure compared to centralized learning. However, it introduces new methods of attacks such as model poisoning, inference/membership attacks and important tradeoffs among communication cost, robustness, and privacy guarantees. We compare classical techniques such as federated averaging with robust aggregation strategies including Trimmed Mean, Krum, Bulyan, and Robust Federated Aggregation, highlighting trade-offs in computational efficiency, scalability, and resilience against compromised clients. Our analysis shows how FL improves data privacy through secure aggregation and differential privacy, and how it deals with threats such as poisoning and inference attacks. A survey of recent patents shows developing interest in communication efficiency, adaptive aggregation and privacy solutions. Overall, FL demonstrates strong potential for privacy-first machine learning, but faces challenges in robustness, communication overhead, and data heterogeneity that define areas of interest for future research

I. EXPERIMENTAL

A. Objectives

- Implement a federated learning system using the Flower framework
- Deploy a label poisoning attack with configurable intensity
- Evaluate the impact of poisoned clients on global model performance
- Compare performance metrics between clean and poisoned clients
- Develop visualization tools for analysis

B. Attack Methods

Two clients were selected for poisoning by label flipping: client 3 and client 17.

Random labels were flipped with random values

Configurable flip rates by percentages of 25, 50, 75, and 100 and stored in a separate directory.

C. Setup

Dataset: previously attempted experiment with the Medical Segmentation Decathlon dataset, however due to following hardware limitations:

Processor: Intel(R) Core(TM) i7-7700HQ CPU

RAM: 24.0 GB

Graphics Card: NVIDIA GeForce GTX 1050 (4 GB), Intel(R) HD Graphics 630 (128 MB)

Other medical datasets were tested and led to similar results as the MSD dataset. Since medical datasets are often 3D/4D which require images to be divided into slices, training and testing with the MSD dataset would take way too long or consume too much memory resulting in worker crashes and other errors. There were about 87 *.nii.gz files per client.

The new dataset chosen is the CIFAR-10 dataset containing 50,000 images (2500 per client (20 clients total)) with 10 different classes. The training and testing dataset was split 80% train: 20% test

Training parameters:

Global Rounds: 5

Training Epochs: 10

Batch Size: 32

Learning Rate: 0.5

Optimizer: Stochastic Gradient Descent

Experiments were conducted under two scenarios:

Scenario 1: No attacking clients, all clients use the original CIFAR-10 images and labels.

Scenario 2: Two poisoned clients, 3, and 17, have certain percentages of labels flipped and labels are randomly chosen.

D. Model Architecture

Input: 32×32×3 RGB Image

Convolutional Block 1:

```
Conv2D(3→32, kernel=3×3, padding=1)
ReLU
Conv2D(32→32, kernel=3×3, padding=1)
ReLU
MaxPool2D(2×2) → 16×16×32
```

Convolutional Block 2:

```
Conv2D(32→64, kernel=3×3, padding=1)
ReLU
Conv2D(64→64, kernel=3×3, padding=1)
ReLU
MaxPool2D(2×2) → 8×8×64
```

Classifier:

```
Flatten → 4096
Linear(4096→256)
ReLU
Linear(256→10)
```

Output: 10-class probabilities

E. Code Architecture

CIFAR_FL_Flower/

task.	# Model, dataset, training/testing functions
client_app.py	# Flower client implementation
server_app.py	# Flower server implementation
custom_strategy.py	# Custom FedAvg with metric logs
pyproject.toml	# Configuration file
data_prep.py	# Data partitioning script
label_flipping.py	# Label poisoning script
visualization.py	# Metrics visualization
CIFAR_clients/	# Clean client data
flipped_labels/	# Poisoned labels
{xxx}flipped/	# percentage flipped labels
results/	# Experimental results
no_attack/	
attack/	

II. RESULTS

A. No Attack Run

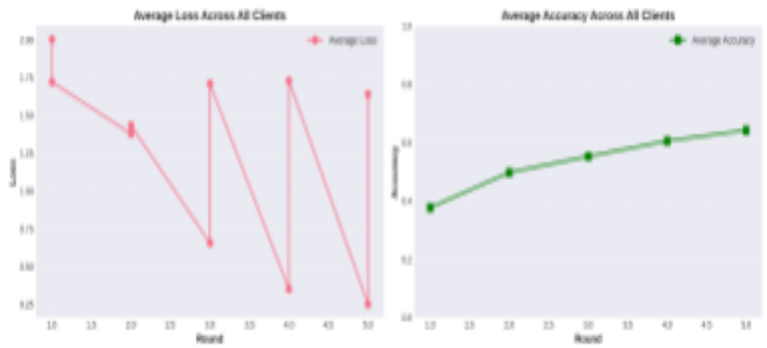


Fig. 1. Average loss per round for train and test (on left) and Average accuracy per round (on right) for a no attack run

As seen in Figure 1 and 2 the average loss shifts much more violently as the number of round increases which may suggest overfitting. Average accuracies increase but begin to plateau quickly capping out at around 63% accuracy.

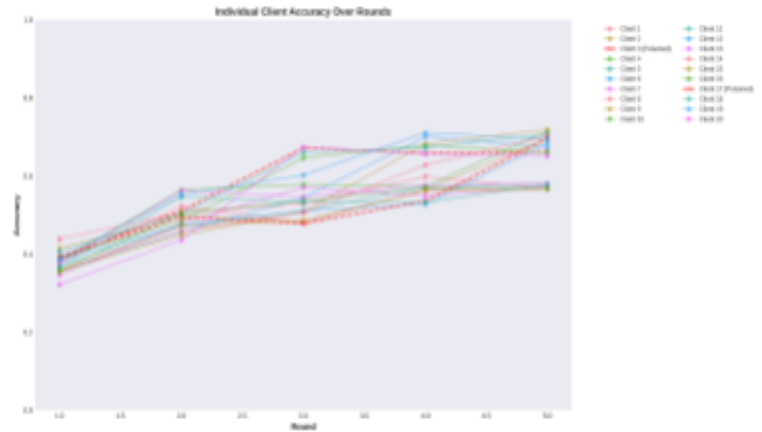


Fig. 2. The individual accuracy of each client per round. Although clients 3 and 17 are labeled as poisoned they were not poisoned during this run.

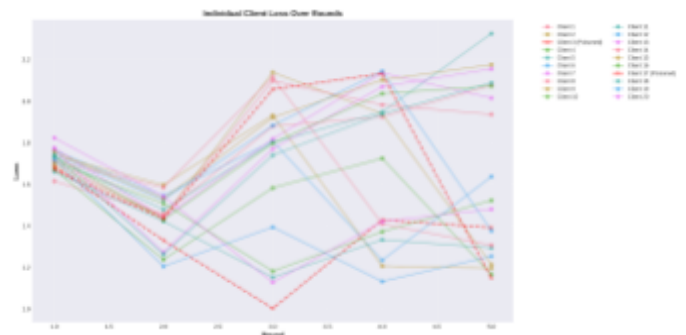


Fig. 3. Individual losses per client over each round

As seen in figures 3 & 4 the losses and accuracies follow the average losses and accuracies in figures 1 and 2.

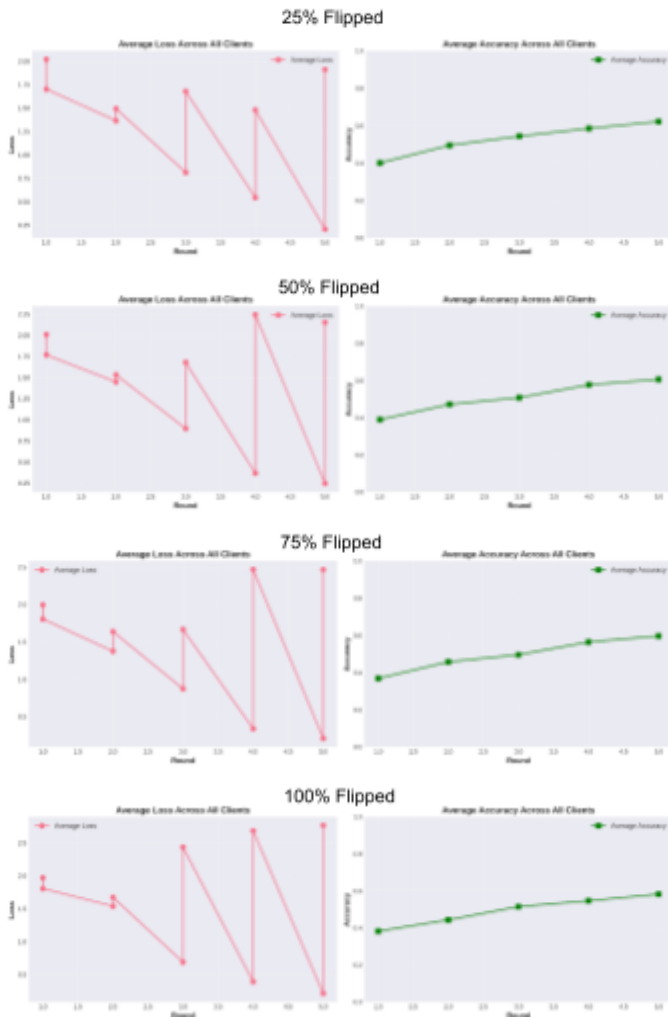


Fig. 4. (25% label flipped)

Fig. 5. (50% label flipped)

Fig. 6. (75% label flipped)

Fig. 7. (100% label flipped)

The average loss during train and testing for each round (on left) and The average accuracies per round (on right)

According to the figures (4, 5, 6, and 7) above the graphs follow the same patterns although some dips are more visible than others most likely because clients are randomly selected for training and the dips represent when a poisoned client is trained/tested on. Additionally, the accuracies do being to decrease earlier than the clean clients. However, I was unable to meet the requirement of attacking run accuracies being below 50%, I believe this is partly due to the limited number of just 2 poisoned clients.

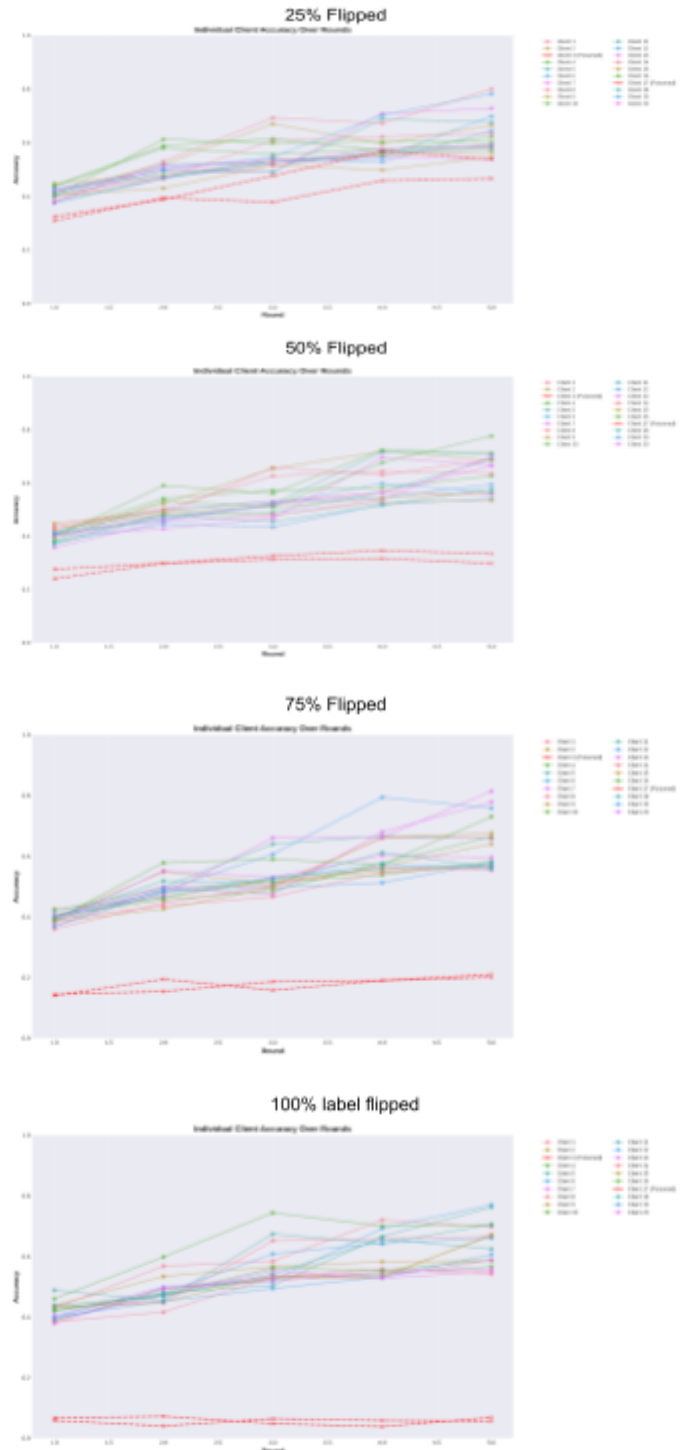


Fig. 8. (25% label flipped)

Fig. 9. (50% label flipped)

Fig. 10. (75% label flipped)

Fig. 11. (100% label flipped)

The Accuracies of each client per round of training and testing

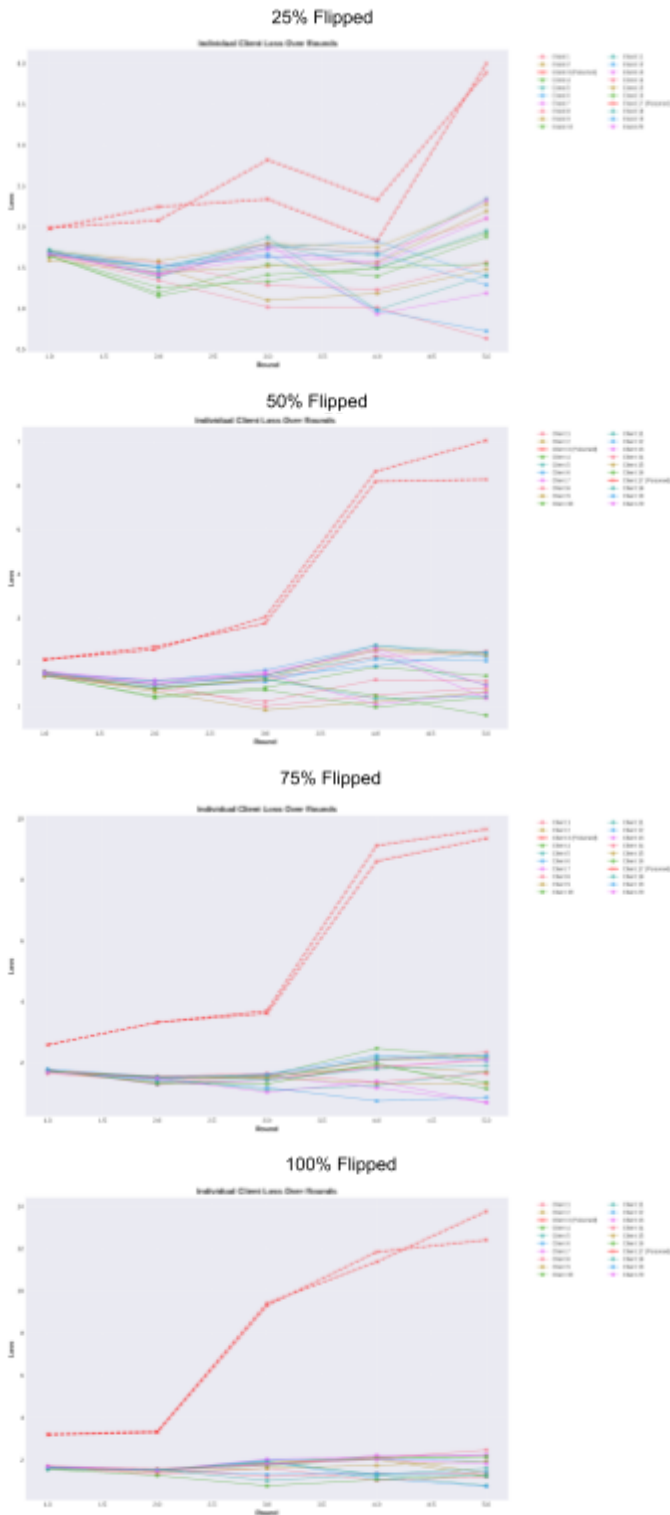


Fig. 12. (25% label flipped)

Fig. 13. (50% label flipped)

Fig. 14. (75% label flipped)

Fig. 15. (100% label flipped)

The losses of each client per round of training and testing

As shown in the figures (8-15) the poisoned clients 3 and 17 do have very low accuracies the greater the flip percentage and very high loss the greater the flip percentage. So it is very likely that the reason why the average accuracy doesn't fall below 50% is because of the low number of poisoned clients.

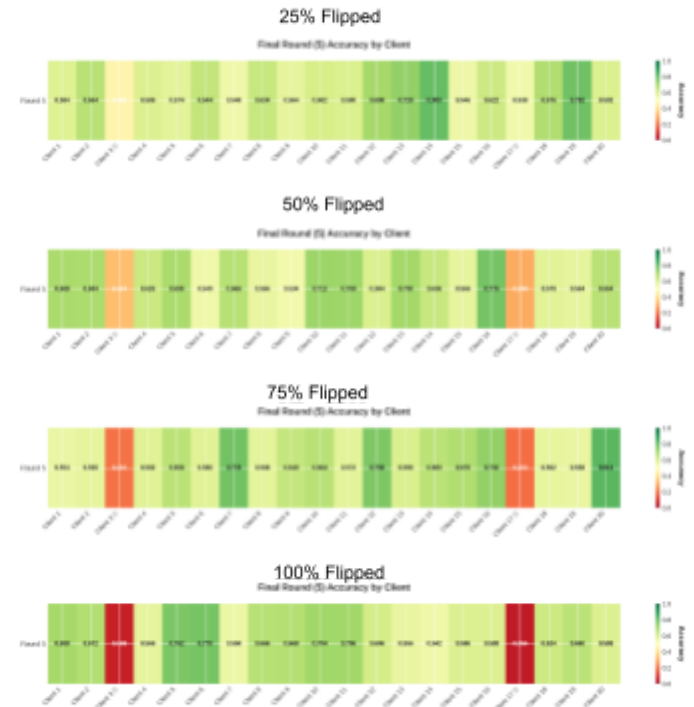


Fig. 16. (25% label flipped)

Fig. 17. (50% label flipped)

Fig. 18. (75% label flipped)

Fig. 19. (100% label flipped)

The final accuracy per client

The final accuracy decreases as the flip percentage increases. However, due to the low number of poisoned clients, its effect on the overall training accuracy is somewhat minimal over a short period of time. However, its effects are evident and if they can implant malicious data over long periods of time it can be damaging to the Federated Learning System. Additionally, due to the obvious difference between the clean and poisoned clients it can be simple to detect and block these poisoned clients although a lower percentage of damage may be harder to detect.

III. WORK DISTRIBUTION

I worked alone on this project although I did use AI to help with graph generation and optimizations with models and data parsing. Running the simulations was definitely the most time consuming, taking over a week, as originally I was using the

MSD dataset and other medical datasets which unfortunately, due to hardware limitations, I was not able to continue with those datasets. The second longest part was when I tried my best to fine tune the learning rates, batch sizes, and the number of epochs and rounds but unfortunately was not able to increase the average accuracy above 63% for clean clients and below 50% for poisoned runs which took around 24 hours.